

BASIC SQL COMMANDS

SQL SELECT Practice for Software Testing

Topics covered: SELECT, WHERE, LIKE, DISTINCT, ORDER BY, LIMIT, BETWEEN, IN, AND, and practical database-testing examples.

Sample users Table

Assume the users table has these columns:

Column	Data Type	Purpose
id	INT	Unique user ID
username	VARCHAR	User's username
email	VARCHAR	User's email address
city	VARCHAR	User's city
age	INT	User's age
signup_date	DATE	Date the user registered

Sample Data

id	username	email	city	age	signup_date
1	amit123	amit@gmail.com	Ahmedabad	21	2026-09-20
2	aisha	aisha@gmail.com	Surat	19	2026-09-18
3	rahul	rahul@gmail.com	Vadodara	27	2026-09-15
4	ankit_dev	ankit@yahoo.com	Ahmedabad	30	2026-09-21
5	priya	priya@gmail.com	Surat	22	2026-09-17
6	rohan	rohan@gmail.com	Mumbai	24	2026-09-10
7	arjun	arjun@gmail.com	Vadodara	20	2026-09-22
8	neha	neha@gmail.com	Ahmedabad	26	2026-09-19
9	akash	akash@gmail.com	Surat	18	2026-09-16
10	meera	meera@gmail.com	Mumbai	31	2026-09-12

1. SELECT + WHERE — Find Users from Ahmedabad

```
SELECT username, email
FROM users
WHERE city = 'Ahmedabad';
```

Explanation: SELECT chooses the username and email columns. FROM users specifies the table. WHERE filters the rows so only users whose city exactly matches Ahmedabad are returned.

username	email
amit123	amit@gmail.com
ankit_dev	ankit@yahoo.com
neha	neha@gmail.com

2. LIKE — Find Usernames Starting with 'a'

```
SELECT username, email
FROM users
WHERE username LIKE 'a%';
```

Explanation: LIKE performs pattern matching. The % wildcard means any number of characters can follow the letter a. For an explicitly case-insensitive comparison, use LOWER(username).

username	email
amit123	amit@gmail.com
aisha	aisha@gmail.com
ankit_dev	ankit@yahoo.com
arjun	arjun@gmail.com
akash	akash@gmail.com

3. DISTINCT + WHERE — Cities with Users Above 25

```
SELECT DISTINCT city
FROM users
WHERE age > 25;
```

Explanation: WHERE first keeps users older than 25. DISTINCT then removes duplicate city names. This is useful when validating a list of unique values in an application filter.

city
Vadodara
Ahmedabad
Mumbai

4. ORDER BY + LIMIT — 5 Most Recent Signups

```
SELECT username, signup_date
FROM users
ORDER BY signup_date DESC
LIMIT 5;
```

Explanation: ORDER BY signup_date DESC sorts from newest to oldest. LIMIT 5 returns only the first five rows. DESC is used because we need the most recently signed-up users.

username	signup_date
arjun	2026-09-22
ankit_dev	2026-09-21
amit123	2026-09-20
neha	2026-09-19
aisha	2026-09-18

5. BETWEEN + IN + AND — Age 18–22 in Surat or Vadodara

```
SELECT username, city, age
FROM users
WHERE age BETWEEN 18 AND 22
AND city IN ('Surat', 'Vadodara');
```

Explanation: BETWEEN 18 AND 22 includes both 18 and 22. IN checks whether city is Surat or Vadodara. AND means both the age and city conditions must be satisfied.

username	city	age
aisha	Surat	19
priya	Surat	22
arjun	Vadodara	20

akash	Surat	18
-------	-------	----

SQL Quick Revision Notes

SQL Concept	Syntax	Purpose
SELECT	SELECT username FROM users;	Select columns
WHERE	WHERE city = 'Ahmedabad'	Filter rows
LIKE	username LIKE 'a%'	Pattern matching
%	'a%'	Any number of characters
DISTINCT	SELECT DISTINCT city	Remove duplicates
ORDER BY	ORDER BY signup_date DESC	Sort results
DESC	DESC	Newest/highest to oldest/lowest
LIMIT	LIMIT 5	Restrict number of rows
BETWEEN	age BETWEEN 18 AND 22	Inclusive range
IN	city IN ('Surat','Vadodara')	Match multiple values
AND	condition1 AND condition2	Both conditions must be true

All 5 Answers Together

```
-- 1. Users from Ahmedabad
SELECT username, email
FROM users
WHERE city = 'Ahmedabad';

-- 2. Usernames starting with 'a'
SELECT username, email
FROM users
WHERE LOWER(username) LIKE 'a%';

-- 3. Distinct cities with users above 25
SELECT DISTINCT city
FROM users
WHERE age > 25;

-- 4. Top 5 most recently signed-up users
SELECT username, signup_date
FROM users
ORDER BY signup_date DESC
LIMIT 5;

-- 5. Users aged 18-22 in Surat or Vadodara
SELECT username, city, age
FROM users
WHERE age BETWEEN 18 AND 22
AND city IN ('Surat', 'Vadodara');
```

Software Tester Practice: These commands help QA testers filter test data, search using patterns, validate unique values, check sorting, verify result limits, and test multiple database conditions.